

FitBuddy: Core Functionalities Development

AI Fitness Plan Generator using Gemini Models | Epic 2 Implementation Document

1. Overview of Core Functionalities (Epic 2)

Epic 2 forms the backbone of *FitBuddy* by implementing core application logic, setting up Gemini model integration, and establishing FastAPI backend routing to manage user input processing. This phase bridges raw user metrics (age, fitness goals, preferences) with Google's generative AI models to yield customized workout regimens.

Key Milestone Objective: Develop the core business logic, implement precise prompt engineering for fitness generation, and configure FastAPI routers to handle data submission and response parsing securely.

2. Core Components & Stories

- **Story 1: Developing Core Functionalities & Prompt Engineering**
 - Initializing the *google-generativeai* SDK.
 - Designing structured prompt templates to guide Gemini models in producing safe, accurate, and multi-day workout/nutrition schedules.
- **Story 2: FastAPI Backend Routing & Input Processing**
 - Implementing *APIRouter* inside *routes.py*.
 - Capturing form data via FastAPI's *Form(...)* parameters and passing them securely to the AI model pipeline.

3. Reference Code: Core Logic & Gemini Integration (*routes.py*)

Below is the implementation showing how user inputs are processed and sent to the Gemini Pro model:

```

from fastapi import APIRouter, Request, Form
from fastapi.templating import Jinja2Templates
import google.generativeai as genai
import os

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.post("/generate")
async def generate_fitness_plan(
    request: Request,
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...)
):
    # Configure Gemini model
    model = genai.GenerativeModel('gemini-pro')

    # Construct prompt for tailored fitness plan
    prompt = (
        f"Act as a professional fitness trainer and nutritionist. "
        f"Create a detailed 7-day workout and diet plan for a {age}-year-old individual "
        f"weighing {weight} kg whose primary goal is {goal}. "
        f"Include daily exercises, sets, reps, and dietary recommendations."
    )

    # Generate content using Gemini
    response = model.generate_content(prompt)
    plan_text = response.text

    # Render results via Jinja2 template
    return templates.TemplateResponse(
        "plan.html",
        {"request": request, "plan": plan_text, "age": age, "goal": goal}
    )

```

4. Error Handling & Validation

To ensure robust core functionality, error handling is implemented around API calls to manage rate limits, invalid API keys, or empty input fields gracefully, ensuring the application does not crash during runtime.

5. Summary

With Epic 2 successfully completed, *FitBuddy* possesses the functional intelligence required to transform user metrics into dynamic fitness solutions.